Following It Is All You Need.

照着走，就够了。

学生缺的不是信息，是下一步。
一个能替他做决定的系统。

NextStop · 概念稿 · 2026-10-29

一、上完一节课，他不知道接下来该干什么

走出来的时候，他手上只有一部手机，和一段说不清多长的空档。他不知道三件事：

该去哪 · 能去哪 · 该干啥

而"哪里有空着的地方"只是第二件事里的一小块 —— 而且是最容易的那块。

现在所有工具都在反复回答这一小块，然后把另外两件半交回给学生：你自己判断哪些能用、你自己决定做什么、你自己算来不来得及。

于是空档被"想"吃掉了。一个 50 分钟的空档，先花 10 分钟决定，再花 5 分钟走过去，还剩 35 分钟 —— 而这 15 分钟不是被任何系统占用的，是被没人替你决定占用的。

二、所以"给更多信息"是没用的

你已经知道楼里到处有空房。你缺的不是信息，是下一步。

这是这个项目唯一要说的判断：信息不会消除迷茫，决定才会。

三、我们做的是指导，不是查询

打开它，屏幕上只有一句：

> 现在去这里。走 5 分钟。10:55 离开。

没有搜索框，没有筛选，没有"请输入时间"。你不选、不比、不算。

后面发生的事才是关键：

- 它是一条序列 —— 这一站是为下一站选的。你 09:47 去哪，决定了 10:50 从哪出发、11:00 到不到得了、下午那段时间还能不能用。
- 它被打断时会修 —— 你晚了 20 分钟、你饿了、那间被人占了、下雨了。它不重新问一遍，只动必须动的那一站。
- 它一周之后开始懂你 —— 你走得比别人慢多少、你几点真的会走、你哪门课会翘、你吃饭愿意花多久。这些没有人愿意填，所以它不问，它看。

四、三件真正难的事

第一件：懂你，但一点都不打扰你。

要懂一个人，最容易的路是问他 —— 而那恰恰是最不该做的。所以这里只有一条硬规则：

> 不问任何你本来不会做的事。

它只从你本来就要做的动作里读：揭开日程、点"我到了"、"换一个"、"我晚了"、"翘课"。没有问卷、没有评分、没有"这次推荐准吗"。第一周它只是看着；第二周起，每学到一个维度就替换一个默认值。它变的永远是一个参数，不是算法 —— 所以它越用越像你，却仍然可以被测试。

第二件：修，而不是重算。

修比重算难。重算只要算一次，修要判断"哪些还成立"。这听起来像工程细节，其实是这个系统之所以不是"自动化查询"的唯一理由：

> 一天四段空隙，拖堂 20 分钟 —— 重算改 3 站，最小修只改 1 站。

这句话现在还是假设。我们先把它跑出来；跑不出来，这个概念就不成立。

第三件：简约而不简单 —— 而且越来越简约。

好系统不该越做越忙。这里的目标是反过来的：

> 操作数只降不升，而这一天的流程越来越让人愿意照着走。

第一版可能要三次点击，第二版两次，第三版一次都不需要 —— 同时它安排的一天要越来越像你会自己安排的那种。"步骤在降、喜欢在升"，这两条曲线交叉的样子，就是这个产品成不成立的证据。

简约不是功能少，是你做的事少：功能可以一直在里面长大，暴露给你的永远只剩一句"现在去这里"。

五、我们已有的那个信息源，在这套东西里的位置

它是一个零件。

它回答三件事里的"能去哪"：此刻哪些地方是开放的、能待多久、有没有插座、多远。

这部分我们已经做完了 ——
数据库、空闲计算引擎、员工变更、评价、测试。它保证指导不会指向一个不存在的地方。

但它不是产品。产品是上面那层：应该去哪，以及被打断之后怎么办。

> 数据决定什么是对的，指导决定你现在做什么。

六、想听你们的三件事

1. "信息不会消除迷茫，决定才会" ——
这个判断你们买账吗？这是整个项目的立论，如果不成立，后面都不用谈。

2. 最小修真的会比重算少改吗？
我猜会，但没验。要不要先花半天把这组数字跑出来，再决定走不走这条路？

3. 时间够吗？ 计划 2.5 天 + 界面 1 天 + 学习 1 天 + 吃饭 0.5 天 ≈ 5 天。如果只有 3 天，先砍哪块？

细节（五种停靠点、meet 的三层、吃饭的弹性规则、翘课怎么建模、AI
的边界、已经被否掉的七个想法）在 设计记录.md。